

Segurança da Informação & Consultoria Web3

Relatório de Auditoria de Segurança

ERC20TrancheToken

Cliente	AxiaDigitalSolutions
Preparado por	HeimdallSec
Data	23/07/2026
Versão do Documento	1.0
Status da Auditoria	Final (correções aplicadas; itens informativos pendentes)

Commit auditado (Git): `9cd96c0b74c0c384d7e4daa934da7e5771494d7d`
https://github.com/AxiaDigitalSolutions/SmartContractsAudit/commit/9cd96c0b74c0c384d7e4daa934da7e5771494d7d

DOCUMENTO CONFIDENCIAL

Sumário

Sumário 2
1. Resumo Executivo 3
1.1 Visão Geral do Escopo 3
1.2 Resumo das Descobertas 3
2. Metodologia de Auditoria 3
2.1 Análise Geral do Contrato 4
3. Descobertas Detalhadas (Findings) 4
[HSEC-ETT-01] Ausência de `whenNotPaused` em Transferências — Pausa Não Bloqueia Transferências de Usuário 4
[HSEC-ETT-02] Risco de Centralização — Admin Único com Todos os Papéis (Single Point of Failure) 5
[HSEC-ETT-03] Mint Pode Criar Tokens para Endereços Não Whitelisted Quando `transferRestricted` Está Ativo 6
[HSEC-ETT-04] Burn de Endereço Não Whitelisted Remove Tokens Sem Verificação de Restrição 7
[HSEC-ETT-05] Ausência de Função `renounceRole` — Possibilidade de Auto-Revogação Irreversível 8
[HSEC-ETT-06] Versão do Solidity com Pragmatismo Amplo — ^0.8.0 Permite Versões Instáveis 8
5. Disclaimer — Análise de Ameaças por Ação 9
6. Termo de Isenção de Responsabilidade (Disclaimer) 9

1. Resumo Executivo

A HeimdallSec foi contratada pela **AxiaDigitalSolutions** para conduzir uma auditoria de segurança no contrato **ERC20TrancheToken**. O objetivo desta auditoria foi identificar vulnerabilidades de segurança, falhas de lógica de negócios e desvios de boas práticas no código-fonte, fornecendo recomendações acionáveis para mitigação.

Este relatório detalha as descobertas feitas durante o período de 23/07/2026. A auditoria combinou análise estática automatizada e revisão manual rigorosa por especialistas em segurança Web3.

O contrato implementa um token ERC-20 com controle de acesso baseado em papéis (RBAC), pausabilidade e suporte a whitelist com restrição de transferência — projetado para representar posições de investidores em operações de crédito/tranches.

1.1 Visão Geral do Escopo

Propriedade	Detalhe
Nome do Projeto	ERC20TrancheToken
Cliente	AxiaDigitalSolutions
Linguagem / Ecossistema	Solidity ^0.8.0 / EVM
Arquivos no Escopo	`ERC20TrancheToken.sol`
Linhas de Código (SLOC)	103 (código-fonte), 7 funções expostas
Dependências	OpenZeppelin (ERC20, AccessControl, Pausable)
Padrões	ERC-20, ERC-165 (via AccessControl)

1.2 Resumo das Descobertas

Durante a auditoria, foram identificados **6** problemas de segurança, distribuídos da seguinte forma. Após a fase de correção, **as descobertas de severidade High, Medium e Low foram corrigidas e verificadas, enquanto os 2 itens Informational / Gas permanecem pendentes (4 de 6 resolvidas)**:

Severidade	Quantidade	Status de Resolução
Critical	0	**0 Resolvidos**
High	1	**1 Resolvidos**
Medium	2	**2 Resolvidos**
Low	1	**1 Resolvidos**
Informational / Gas	2	**0 Resolvidos**

2. Metodologia de Auditoria

Nossa auditoria segue uma metodologia estruturada baseada em padrões de mercado, incluindo as diretrizes do **EEA EthTrust Security Levels** e o **OWASP Smart Contract Security Verification Standard (SCSVS)**.

O processo foi executado nas seguintes fases:

1. **Modelagem de Ameaças:** Mapeamento de atores (DEFAULT_ADMIN_ROLE, MINTER_ROLE, BURNER_ROLE, PAUSER_ROLE), privilégios e vetores de ataque.
2. **Análise Automatizada:** Varredura conceitual com padrões de segurança conhecidos (reentrância, controle de acesso, validação de entrada, CEI).

3. **Revisão Manual:** Leitura linha a linha focada em falhas de lógica de negócios, controle de acesso e validade da whitelist.

As vulnerabilidades são classificadas utilizando o sistema da **Immunefi (v2.3)**, baseando-se no impacto potencial (ex: perda de fundos, DoS) e na probabilidade de exploração.

2.1 Análise Geral do Contrato

O contrato **ERC20TrancheToken** estende `ERC20`, `AccessControl` e `Pausable` da OpenZeppelin, fornecendo:

- **Mint restrito:** Apenas `MINTER_ROLE` pode criar novos tokens, e apenas quando o contrato não está pausado.
- **Burn restrito:** Apenas `BURNER_ROLE` pode queimar tokens, e apenas quando o contrato não está pausado.
- **Pausabilidade:** `PAUSER_ROLE` pode pausar/despausar operações de mint e burn.
- **Whitelist:** `DEFAULT_ADMIN_ROLE` pode gerenciar uma lista branca de endereços autorizados a transferir tokens.
- **Restrição de transferência:** Quando `transferRestricted == true` (padrão), apenas endereços na whitelist podem transferir tokens entre si.

Funções Expostas:

Função	State Mutability	Privilégio	Descrição
`mint(address, uint256)`	`nonpayable`	MINTER_ROLE	Cria novos tokens para um endereço
`burn(address, uint256)`	`nonpayable`	BURNER_ROLE	Queima tokens de um endereço
`pause()`	`nonpayable`	PAUSER_ROLE	Pausa operações de mint/burn
`unpause()`	`nonpayable`	PAUSER_ROLE	Despausa operações
`updateWhitelist(address, bool)`	`nonpayable`	DEFAULT_ADMIN_ROLE	Adiciona/remove da whitelist
`setTransferRestricted(bool)`	`nonpayable`	DEFAULT_ADMIN_ROLE	Ativa/desativa restrição de transferência

3. Descobertas Detalhadas (Findings)

[HSEC-ETT-01] Ausência de `whenNotPaused` em Transferências — Pausa Não Bloqueia Transferências de Usuário

Severidade: High **Status: Resolved Localização:** `ERC20TrancheToken.sol`, Função `_beforeTokenTransfer()` (Linhas `92-102`)

Descrição: O modificador `whenNotPaused` é aplicado apenas às funções `mint()` e `burn()`, mas NÃO é verificado nas transferências comuns (`transfer()` e `transferFrom()` herdadas do ERC-20). O hook `_beforeTokenTransfer()` não possui nenhuma verificação de pausa.

Isso significa que, quando o contrato está pausado:

- **Mint e burn são bloqueados** ✓ (correto)
- **Transferências de usuário NÃO são bloqueadas** ✗ (risco)

Em cenários de emergência (comprometimento, migração forçada), a intenção de pausar o contrato é congelar todos os movimentos de token — mas as transferências P2P continuam funcionando normalmente.

Cenário de Exploração (Impacto):

1. Equipe detecta atividade maliciosa e chama `pause()`.
2. Atacante ainda consegue transferir tokens para outra wallet (dificultando recuperação).
3. Sistemas de compliance e auditoria off-chain podem ser enganados.

Recomendação de Mitigação: Adicionar verificação `whenNotPaused` dentro de `_beforeTokenTransfer()` para bloquear também transferências:

```
function _beforeTokenTransfer(
    address from,
    address to,
    uint256 amount
) internal virtual override {
    super._beforeTokenTransfer(from, to, amount);

    if (from != address(0) && to != address(0)) {
        _requireNotPaused();  // ← NOVA verificação
        if (transferRestricted) {
            require(whitelist[from] || whitelist[to],
                "ERC20TrancheToken: transfer restricted to whitelisted addresses");
        }
    }
}
```

Correção Aplicada: A verificação `_requireNotPaused()` foi adicionada dentro do hook `_beforeTokenTransfer()`, de modo que, quando o contrato está pausado, todas as movimentações de token — incluindo transferências P2P (`transfer()` e `transferFrom()`) — passam a ser bloqueadas, além de mint e burn. A pausa agora congela integralmente o token, atendendo à intenção de resposta a emergências.

Verificação da Correção: ✓ Verificado. A equipe de auditoria confirmou que o hook `_beforeTokenTransfer()` executa `_requireNotPaused()` para transferências entre usuários; com o contrato pausado, tentativas de `transfer`/`transferFrom` revertem corretamente.

Resposta da Equipe (Opcional): Correção implementada conforme recomendado: pausa passou a bloquear também as transferências de usuário.

[HSEC-ETT-02] Risco de Centralização — Admin Único com Todos os Papéis (Single Point of Failure)

Severidade: Medium **Status: Resolved Localização:** `ERC20TrancheToken.sol`, Constructor (Linhas `29-39`)

Descrição: O construtor concede `DEFAULT_ADMIN_ROLE`, `MINTER_ROLE`, `BURNER_ROLE` e `PAUSER_ROLE` a um único endereço `admin`. Isso significa que uma única chave privada tem controle total sobre:

- Criar tokens ilimitados (MINTER_ROLE → mint)
- Queimar tokens de qualquer endereço (BURNER_ROLE → burn)
- Pausar/despausar o contrato (PAUSER_ROLE)
- Gerenciar whitelist (DEFAULT_ADMIN_ROLE)
- Ativar/desativar restrição de transferência (DEFAULT_ADMIN_ROLE)

Se a chave do admin for comprometida, um atacante pode drenar o contrato (mint ilimitado + transfer para si mesmo) e manipular todas as restrições.

Cenário de Exploração (Impacto):

1. Chave privada do admin é comprometida via phishing ou vazamento.
2. Atacante faz mint de tokens ilimitados para seu endereço.
3. Atacante adiciona seu endereço à whitelist.
4. Atacante desativa restrição de transferência.
5. Tokens podem ser vendidos/dumpados no mercado, causando perda financeira total para investidores legítimos.

Recomendação de Mitigação:

1. **Separar papéis em diferentes EOAs ou multisigs:**
- `DEFAULT_ADMIN_ROLE` e `PAUSER_ROLE` em um Multisig (Safe)
- `MINTER_ROLE` e `BURNER_ROLE` em contas operacionais com menor privilégio
1. **Usar contrato Multisig** (Safe) como admin principal.
2. **Adicionar timelock** para operações críticas (`updateWhitelist`, `setTransferRestricted`):

```
mapping(bytes32 => uint256) public pendingChanges;

function scheduleWhitelistUpdate(address account, bool isWhitelisted)
    external onlyRole(DEFAULT_ADMIN_ROLE)
{
    bytes32 key = keccak256(abi.encodePacked("whitelist", account));
    pendingChanges[key] = block.timestamp + 2 days;
    emit ChangeQueued("whitelist", account, isWhitelisted, pendingChanges[key]);
}

function executeWhitelistUpdate(address account, bool isWhitelisted)
    external onlyRole(DEFAULT_ADMIN_ROLE)
{
    bytes32 key = keccak256(abi.encodePacked("whitelist", account));
    require(block.timestamp >= pendingChanges[key], "Timelock not expired");
    whitelist[account] = isWhitelisted;
    emit WhitelistUpdated(account, isWhitelisted);
}
```

Correção Aplicada: Os papéis foram segregados para eliminar o ponto único de falha: `DEFAULT_ADMIN_ROLE` e `PAUSER_ROLE` passaram a ser controlados por um Multisig (Safe), enquanto `MINTER_ROLE` e `BURNER_ROLE` foram atribuídos a contas operacionais de menor privilégio. Adicionalmente, um mecanismo de timelock foi introduzido para as operações críticas (`updateWhitelist`, `setTransferRestricted`), exigindo agendamento prévio e período de espera antes da execução.

Verificação da Correção: ✓ Verificado. A equipe de auditoria confirmou a separação de papéis entre o multisig e as contas operacionais e a presença do timelock nas operações administrativas críticas. O comprometimento de uma única chave não é mais suficiente para drenar o contrato ou desativar as proteções.

Resposta da Equipe (Opcional): Correção implementada: papéis distribuídos entre multisig e contas operacionais, com timelock nas ações administrativas críticas.

[HSEC-ETT-03] Mint Pode Criar Tokens para Endereços Não Whitelisted Quando `transferRestricted` Está Ativo

Severidade: Medium **Status: Resolved Localização:** `ERC20TrancheToken.sol`, Função `mint()` (Linhas `46-48`)

Descrição: A função `mint()` permite que `MINTER_ROLE` crie tokens para QUALQUER endereço, independentemente de ele estar na whitelist ou não. Quando `transferRestricted == true`, o destinatário do mint fica com tokens que não pode transferir (já que não está na whitelist), efetivamente criando tokens "bloqueados".

Embora isso possa ser intencional (ex: liquidar tokens que depois serão distribuídos por um agente), a falta de verificação pode levar a:

- Tokens mintados para endereços incorretos que não podem ser transferidos.
- Necessidade de o admin adicionar o endereço à whitelist como segunda transação.
- Perda permanente de tokens se o admin perder acesso (tokens mintados para `address(0)` ou contratos sem mecanismo de saída).

Cenário de Exploração (Impacto):

1. `MINTER_ROLE` faz mint para um novo investidor antes de adicioná-lo à whitelist.
2. Investidor recebe tokens mas não pode movê-los.
3. Admin precisa de outra transação para adicionar whitelist.

4. Em cenários de erro humano, tokens podem ficar presos permanentemente.

Recomendação de Mitigação: Adicionar verificação no mint para garantir que o destinatário está na whitelist quando `transferRestricted == true`, ou documentar claramente que o mint SEMPRE deve ser precedido (ou seguido) pela adição à whitelist:

```
function mint(address to, uint256 amount) external onlyRole(MINTER_ROLE) whenNotPaused {
    if (transferRestricted) {
        require(whitelist[to], "ERC20TrancheToken: recipient must be whitelisted");
    }
    _mint(to, amount);
}
```

Alternativa: Se o fluxo de negócios exige mint antes da whitelist (ex: mint em lote seguido de distribuição ao mercado), adicionar documentação clara no NatSpec.

Correção Aplicada: A função `mint()` passou a validar, quando `transferRestricted == true`, que o destinatário está na whitelist antes de criar tokens (`require(whitelist[to], "ERC20TrancheToken: recipient must be whitelisted")`). Isso impede a criação de tokens "bloqueados" em endereços não autorizados e elimina o risco de tokens presos por erro humano.

Verificação da Correção: ✓ Verificado. A equipe de auditoria confirmou que o mint reverte ao tentar criar tokens para endereços fora da whitelist enquanto a restrição de transferência está ativa, e opera normalmente para destinatários whitelisted.

Resposta da Equipe (Opcional): Correção implementada: mint passou a exigir destinatário na whitelist quando `transferRestricted` está ativo.

[HSEC-ETT-04] Burn de Endereço Não Whitelisted Remove Tokens Sem Verificação de Restrição

Severidade: Low **Status: Resolved Localização:** `ERC20TrancheToken.sol`, Função `burn()` (Linhas `55-57`)

Descrição: A função `burn()` permite que `BURNER_ROLE` queime tokens de qualquer endereço, inclusive endereços que NÃO estão na whitelist. Em cenários onde `transferRestricted == true`, isso permite que tokens sejam removidos de endereços "não autorizados" sem que esses endereços possam se defender.

Embora o burn de um endereço só seja possível se o chamador tiver `BURNER_ROLE` (privilégio restrito), não há verificação de que o endereço de origem consinta com a queima. Isso pode ser usado para:

- Queimar tokens de investidores como medida punitiva.
- Reduzir o supply total sem o consentimento dos holders.
- Criar cenários de DoS para investidores que mantêm saldos não whitelisted.

Cenário de Exploração (Impacto):

1. `BURNER_ROLE` (ou admin com todos os papéis) queima tokens de um investidor.
2. Investidor perde seus tokens sem possibilidade de contestação on-chain.
3. Embora o `BURNER_ROLE` seja restrito, a centralização de poder (finding HSEC-ETT-02) torna este um vetor de abuso viável.

Recomendação de Mitigação:

1. Documentar explicitamente que `BURNER_ROLE` tem poder de queima unilateral.
2. Emitir eventos com informações detalhadas para auditoria off-chain.
3. Considerar exigir que o endereço de origem aprove a queima (via `approve` + `burnFrom`):

```
// Alternativa: burnFrom com allowance
function burnFrom(address account, uint256 amount) external onlyRole(BURNER_ROLE) whenNotPaused {
    uint256 currentAllowance = allowance(account, _msgSender());
    require(currentAllowance >= amount, "ERC20: insufficient allowance");
    _approve(account, _msgSender(), currentAllowance - amount);
    _burn(account, amount);
}
```

Correção Aplicada: O poder de queima unilateral do `BURNER_ROLE` foi documentado explicitamente no NatSpec e o fluxo baseado em consentimento foi adotado: a queima passou a exigir aprovação prévia do titular via `approve` + `burnFrom`, consumindo a allowance do endereço de origem. Eventos detalhados foram emitidos para permitir auditoria off-chain de cada queima.

Verificação da Correção: ✓ Verificado. A equipe de auditoria confirmou que o `burnFrom` valida e decrementa a allowance do titular antes de queimar, impedindo destruição de tokens sem consentimento on-chain, e que os eventos de auditoria são emitidos.

Resposta da Equipe (Opcional): Correção implementada: queima passou a depender de allowance do titular (`burnFrom`), com eventos de auditoria e documentação do privilégio.

[HSEC-ETT-05] Ausência de Função `renounceRole` — Possibilidade de Auto-Revogação Irreversível

Severidade: Informational **Status: Unresolved Localização:** `ERC20TrancheToken.sol`, Herda `AccessControl` da OpenZeppelin

Descrição: O contrato herda `AccessControl` da OpenZeppelin, que inclui a função `renounceRole(bytes32 role, address account)`. Esta função permite que um endereço abra mão de seu próprio papel de forma irreversível.

Se o `admin` (DEFAULT_ADMIN_ROLE) acidentalmente chamar `renounceRole(DEFAULT_ADMIN_ROLE, adminAddress)`:

- Ninguém mais pode gerenciar whitelist.
- Ninguém mais pode ativar/desativar restrição de transferência.
- Ninguém mais pode adicionar novos minters/burners/pausers.
- O contrato se torna permanentemente imutável (ou inutilizável, caso roles restantes sejam perdidos).

Recomendação de Mitigação:

1. **Sobrescrever `renounceRole` para bloquear ou adicionar timelock:**

```
function renounceRole(bytes32 role, address account)
    public virtual override
{
    if (role == DEFAULT_ADMIN_ROLE) {
        revert("ERC20TrancheToken: cannot renounce DEFAULT_ADMIN_ROLE");
    }
    super.renounceRole(role, account);
}
```

1. **Alternativa:** Não bloquear, mas documentar o risco para o operador.

Resposta da Equipe: Pendente — correção ainda não aplicada pela equipe.

[HSEC-ETT-06] Versão do Solidity com Pragmatismo Amplo — ^0.8.0 Permite Versões Instáveis

Severidade: Informational **Status: Unresolved Localização:** `ERC20TrancheToken.sol`, Linha `2`

Descrição: O contrato utiliza `pragma solidity ^0.8.0`, que permite qualquer versão 0.8.x. Embora versões recentes do Solidity 0.8.x sejam estáveis, o pragmatismo amplo pode levar a:

- Compilação com versões mais antigas que contêm bugs (ex: 0.8.0-0.8.4 têm problemas conhecidos com AbiEncoder e otimizações).
- Diferenças de comportamento entre versões (raras, mas existentes).
- Inconsistência entre ambientes de teste e produção.

Recomendação de Mitigação: Fixar a versão para uma específica e testada, como `^0.8.20` ou `^0.8.24`:

```
pragma solidity ^0.8.24;
```

Resposta da Equipe: Pendente — correção ainda não aplicada pela equipe.

5. Disclaimer — Análise de Ameaças por Ação

Abaixo, um resumo do modelo de ameaças por ação privilegiada:

Ação	Quem Pode Fazer	Impacto Potencial	Mitigação Recomendada
Mint	MINTER_ROLE	Inflação do supply, criação de tokens para atacante	Rate limiting, multisig
Burn	BURNER_ROLE	Destruição unilateral de tokens de terceiros	Consentimento do holder
Pause	PAUSER_ROLE	DoS temporário de mint/burn (não de transfers)	Extender para transfers
UpdateWhitelist	DEFAULT_ADMIN_ROLE	Exclusão de investidores legítimos	Timelock + multisig
SetTransferRestricted	DEFAULT_ADMIN_ROLE	Desligamento total de proteções	Timelock + multisig
RenounceRole	Qualquer role holder	Perda irreversível de funcionalidades	Bloquear renúncia de admin

6. Termo de Isenção de Responsabilidade (Disclaimer)

Este relatório de auditoria não constitui um endosso, garantia ou certificação absoluta de que o código auditado está livre de vulnerabilidades ou falhas. A auditoria representa um esforço de tempo limitado (time-boxed) para identificar o máximo possível de problemas de segurança utilizando o conhecimento e as ferramentas disponíveis no momento da execução.

A HeimdallSec não se responsabiliza por perdas financeiras, danos operacionais ou explorações futuras que possam ocorrer no protocolo.

gov.br
Documento assinado digitalmente
KLEBER AQUINO DOS SANTOS JUNIOR
Data: 24/07/2026 21:18:39-0300
Verifique em https://validar.iti.gov.br